

A REPORT OF ONE MONTH TRAINING
at
GURU NANAK DEV ENGINEERING COLLEGE

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD
OF THE DEGREE OF

BACHELOR OF TECHNOLOGY
(Computer Science Engineering)



JUNE-JULY, 2025

SUBMITTED BY:
NAME: JYOTI KUMARI VERMA
UNIVERSITY ROLL NO: 2302573

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
GURU NANAK DEV ENGINEERING COLLEGE LUDHIANA
(An Autonomous College Under UGC ACT)

CERTIFICATE



CANDIDATE’S DECLARATION

I “Jyoti Kumari Verma” hereby declare that I have undertaken one month training at “Guru Nanak Dev Engineering college” during a period from 23 June 2025 to 23 July 2025 in partial fulfillment of requirements for the award of degree of B.Tech (Computer Science Engineering) at GURU NANAK DEV ENGINEERING COLLEGE, LUDHIANA. The work which is being presented in the training report submitted to Department of Computer Science Engineering at GURU NANAK DEV ENGINEERING COLLEGE, LUDHIANA is an authentic record of training work.

Signature of the Student

The one month industrial training Viva-Voce Examination of Jyoti Kumari Verma has been held on _____ and accepted.

Signature of Internal Examiner

Signature of External Examiner

ABSTRACT

This report details the work and learning undertaken during a comprehensive one-month industrial training program focused on Prompt Engineering. The training provided a deep dive into the principles and practices of designing effective prompts to interact with and control large language models (LLMs). The curriculum began with the fundamentals of generative AI and the critical role of prompt engineering, covering Python basics, different types of prompts, and best practices for structuring queries.

A significant portion of the training involved hands-on experience with leading AI platforms, including Google AI Studio and the OpenAI Playground. Practical sessions explored model parameter settings, multimodal interactions (text, speech, and audio), and the integration of Gemini and OpenAI models into applications. The training culminated in the development of real-time projects, such as an automated invoice processing system and a video analysis application, using advanced tools like Langflow and Retrieval-Augmented Generation (RAG) agents. This report documents the methodologies learned, the projects undertaken, and the key outcomes, demonstrating a proficient understanding of modern prompt engineering techniques and their practical application in building intelligent systems.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me throughout my industrial training.

First and foremost, I am deeply grateful to **Guru Nanak Dev Engineering college** for providing me with the opportunity to undertake this training in the field of Prompt Engineering. I extend my heartfelt thanks to my industrial mentor, **Pf. Jaswant Singh Taur**, for their invaluable guidance, encouragement, and insightful feedback during the entire training period. Their expertise and support were instrumental in the successful completion of my projects.

I would also like to thank the Department of Computer Science Engineering at Guru Nanak Dev Engineering College, Ludhiana. I am particularly thankful to Dr. **Kiran Jyoti**, Head of the Department, and my faculty coordinator, **Pf. Kapil Sharma**, for their continuous support and for providing the necessary academic foundation for this training.

Finally, I would like to thank my family and friends for their unwavering encouragement and support throughout this journey.

Jyoti Kumari Verma

2302573

LIST OF FIGURES

1.1	<i>A conceptual diagram showing how a well-structured prompt (Input) guides a Large Language Model (LLM) to produce a desired, accurate response (Output).</i>	2
1.2	<i>A simplified view of the LLM architecture, highlighting the key stages from input text to generated output.</i>	5
1.3	<i>An example of the Langflow user interface, a visual tool used for building complex LLM applications by connecting different components.</i>	8
2.1	<i>An image of the Golden Temple generated programmatically from a text prompt. .</i>	11
2.2	<i>A screenshot showing the output of the Automated Web Summarizer, displaying the summary generated from the GNDEC college website.</i>	12
2.3	<i>A screenshot of the Google AI Studio interface, showing the stream and chat functionalities for real-time interaction with Gemini models.</i>	14
2.4	<i>The menu-driven interface of the voice assistant, showing the options for Text-to-Speech, Speech-to-Text, and Speech-to-Speech.</i>	14
2.5	<i>The terminal output of the fully integrated Speech-to-Speech function, showing the transcribed user query and the LLM's generated response.</i>	16
2.6	<i>A diagram illustrating the workflow of an AI Agent using function calling to interact with external tools.</i>	18
2.7	<i>The secure login page for the AI Research Assistant, built using Flask.</i>	19
2.8	<i>The main dashboard of the AI Assistant after a successful login, showing the various summarization and chat features available to the user.</i>	20
2.9	<i>The user's summary history page, demonstrating the application's ability to save and retrieve past results from the SQLite database.</i>	20

LIST OF TABLES

1.1	<i>Comparison of Foundational Prompting Techniques</i>	6
-----	--	---

CONTENTS

Certificate	i
Candidate's Declaration	ii
Abstract	iii
Acknowledgement	iv
List of Figures	v
List of Figures	v
List of Tables	vi
List of Tables	vi
1 Introduction	1
1.1 Background of the Topic	1
1.1.1 What are Prompts?	2
1.1.2 The Importance of Prompt Engineering	2
1.2 Theoretical Explanation	3
1.2.1 Fundamentals of Generative AI	3
1.2.2 Types of Prompts and Prompting Techniques	3
1.2.3 How Prompt Engineering Works	4
1.2.4 The LLM Working System: From Prompt to Output	5
1.2.5 Foundational Prompting Techniques	6
1.2.6 Model Parameter Settings	6
1.2.7 Advanced Prompting Architectures	7

1.3	Software and Hardware Tools Learned	7
1.3.1	Key Applications	8
2	Training Work Undertaken	10
2.1	Week 1: Foundational Skills and Concepts	10
2.2	Project: Automated Web Content Summarizer	11
2.2.1	Methodology and Learning	12
2.3	Week 2: Prototyping with Google AI Studio and Multimodality	13
2.3.1	Exploring the Google AI Studio Environment	13
2.4	Week 2: Developing a Multimodal Voice Assistant	13
2.4.1	Project Architecture and Methodology	14
2.4.2	Core Functionality Implementation	14
2.4.3	Key Learnings and Challenges	15
2.5	Week 3: Advanced Techniques with OpenAI and Agents	16
2.5.1	Project: Supervised Model Fine-Tuning	16
2.5.2	Developing AI Agents with LangChain and Function Calling	17
2.6	Week 4: Capstone Project - The AI Research Assistant	18
2.6.1	System Architecture and Features	19
2.6.2	Methodology: The RAG Pipeline for PDF Chat	20
3	Results and Discussions	21
3.1	Performance of the Multi-Source Summarizer	21
3.1.1	Results	21
3.2	Performance of the RAG System (PDF Chat)	22
3.2.1	Results	22
3.3	Discussion of Challenges and Key Learnings	22
3.3.1	Challenges	23
3.3.2	Key Learnings	23
4	Conclusion and Future Scope	25
4.1	Conclusion	25
4.2	Future Scope	26

References	27
-------------------	-----------

References	27
-------------------	-----------

CHAPTER 1

INTRODUCTION

1.1 Background of the Topic

The last decade has witnessed a monumental transformation in the field of Artificial Intelligence, primarily driven by advancements in Natural Language Processing (NLP). The development and scaling of the transformer architecture have given rise to a new class of models known as Large Language Models (LLMs). These models, such as OpenAI's Generative Pre-trained Transformer (GPT) series and Google's Gemini, are trained on petabytes of text and code data from the internet, enabling them to generate human-like text, translate languages, write different kinds of creative content, and answer questions in an informative way.

This leap in capability has shifted the paradigm of human-computer interaction. Previously, interacting with complex systems required structured programming languages or rigid command-line inputs. With LLMs, the primary interface is natural language itself. This accessibility, however, introduces a new challenge: how to communicate instructions to these models effectively and reliably. This challenge is the domain of Prompt Engineering.

Prompt Engineering is the discipline of designing, refining, and optimizing input text (prompts) to steer an LLM towards a specific, desired outcome. The quality, relevance, and safety of an LLM's output are directly and profoundly dependent on the quality of the prompt it receives. A well-crafted prompt can unlock the model's vast knowledge and reasoning capabilities, while a poorly constructed one can lead to generic, factually incorrect (a phenomenon known as "hallucination"), or even harmful responses. Therefore, this training was undertaken to gain a deep, practical understanding of this critical discipline, moving beyond casual interaction to a systematic approach for building robust and valuable AI-powered applications.

1.1.1 What are Prompts?

Prompts are pieces of text that provide context and guidance to machine learning models, particularly in the domain of Natural Language Processing (NLP). They are the primary mechanism through which humans communicate tasks and instructions to an AI. A well-designed prompt is crucial for generating relevant and accurate outputs that align with user expectations. The effectiveness of a prompt is determined by several factors:

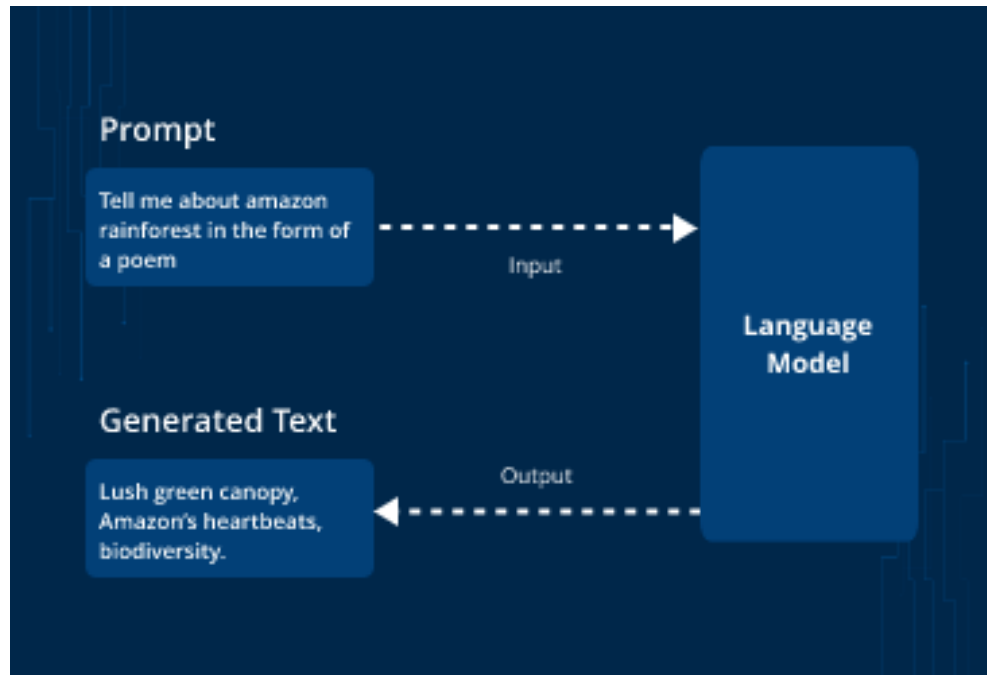


Figure 1.1. A conceptual diagram showing how a well-structured prompt (Input) guides a Large Language Model (LLM) to produce a desired, accurate response (Output).

- **Guidance:** It should offer additional guidance to steer the model's response.
- **Specificity:** It must be specific enough to avoid ambiguity, yet not so overly detailed that it confuses the model.
- **Clarity of Intent:** The user's goal and purpose must be made clear to ensure the model generates content in the correct context.

1.1.2 The Importance of Prompt Engineering

Prompt engineering is the systematic process of designing, refining, and optimizing these prompts. Its importance cannot be overstated, as it directly impacts the performance and reliability of LLM-based systems. Key benefits include:

- **Improved Interpretability:** Specific formats and clear instructions in a prompt allow the model to better interpret the requirements of a task.
- **Higher Accuracy:** Detailed and well-structured prompts lead to outputs that more closely match the desired outcome, reducing errors and iterations.
- **Enhanced Model Training:** In many systems, the interactions via prompts are used to further train and refine the model, meaning better prompts contribute to a better-trained model for future tasks.

1.2 Theoretical Explanation

The training curriculum was structured to build a strong theoretical foundation before moving to practical implementation. This involved understanding not just what prompts to write, but why they work.

1.2.1 Fundamentals of Generative AI

The core of modern LLMs is the transformer architecture, which relies on a mechanism called "self-attention" to weigh the importance of different words in the input text. This allows the model to grasp context, nuance, and long-range dependencies in language. The training began by demystifying this architecture and contrasting the objective of generative AI with its counterpart, discriminative AI. While discriminative models are trained to classify or predict labels from a fixed set of possibilities (e.g., identifying spam in an email), generative models learn the underlying distribution of the data to create entirely new, synthetic data that resembles the training data. This fundamental difference explains why LLMs can "create" new text rather than just categorizing existing text.

1.2.2 Types of Prompts and Prompting Techniques

A significant portion of the theoretical learning was dedicated to a taxonomy of prompting strategies. Each strategy is a tool designed for a specific type of task:

- **Zero-Shot Prompting:** This is the most basic form of interaction, where the model is asked to perform a task without any prior examples. It leverages the model's pre-trained

knowledge. For instance, a simple zero-shot prompt would be: "Summarize the following article into three key points." This technique is effective for general tasks but lacks control for specific output formats.

- **Few-Shot Prompting:** To gain more control, few-shot prompting provides the model with a small number of examples (typically 1 to 5) of the desired input-output pattern. This helps the model understand the expected format, tone, and structure. For example, to perform sentiment analysis, one might provide: "Text: 'I loved the movie!' - Sentiment: Positive. Text: 'The service was terrible.' - Sentiment: Negative. Text: 'The product is okay.' - Sentiment:". The model learns the pattern and completes the final entry.
- **Chain-of-Thought (CoT) Prompting:** For tasks requiring logical reasoning or multi-step calculations, CoT prompting is a powerful technique. Instead of asking for a direct answer, the prompt instructs the model to "think step by step" or to "show its work." This forces the model to externalize its reasoning process, often leading to more accurate results for complex problems. For example, a math problem would be accompanied by a few-shot example where the solution is worked out in logical steps.

1.2.3 How Prompt Engineering Works

The process of prompt engineering can be compared to instructing a talented but inexperienced assistant. Clear and precise instructions are necessary to achieve the desired result. The workflow involves three key stages:

1. **Crafting the Prompt:** The initial step is to design a prompt that clearly specifies what the LLM should do. This can range from a simple question to a complex statement with examples and constraints. The choice of wording, phrasing, and context are all critical variables.
2. **Understanding the LLM:** Different LLMs may respond differently to the same prompt. Part of the engineering process is understanding the nuances of the specific model being used and selecting the appropriate prompting technique.
3. **Refining the Prompt:** Prompt engineering is often an iterative, trial-and-error process. The initial output from the LLM is analyzed, and the prompt is tweaked and refined based

on the results to progressively improve the quality of the response.

1.2.4 The LLM Working System: From Prompt to Output

To effectively engineer prompts, it is essential to understand the basic working mechanism of an LLM. Modern models are built upon the **transformer architecture**, which uses a powerful concept called the **self-attention mechanism**. This allows the model to weigh the importance of different words within the prompt to understand context and nuance. For instance, in the sentence, "The robot picked up the red ball because it was heavy," the self-attention mechanism helps the model determine that "it" refers to the "ball," not the "robot." [1]

The generative process itself is **autoregressive**, as illustrated in Figure 1.2. When a user provides a prompt (Input Text), it is broken down into numerical representations (Tokens and Embeddings) and processed by an Encoder. The self-attention mechanism then allows the model to understand the context. Finally, a Decoder generates the Output Text one word at a time, with each new word being influenced by the words that came before it. The user's prompt is therefore the critical starting context for this entire generative chain, setting the direction and constraining the model's predictions.

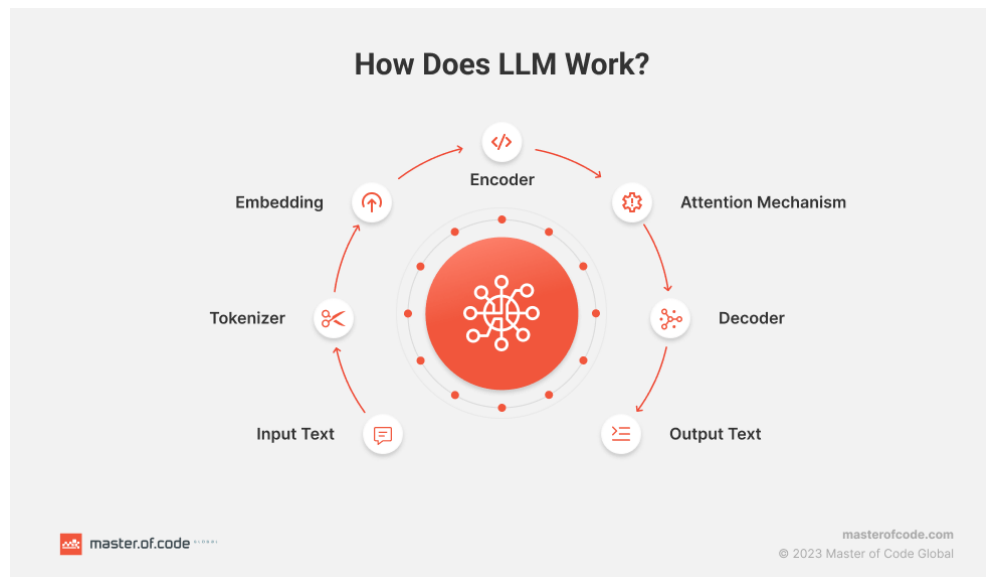


Figure 1.2. A simplified view of the LLM architecture, highlighting the key stages from input text to generated output.

1.2.5 Foundational Prompting Techniques

The training covered a range of foundational and advanced techniques for controlling LLM behavior. These techniques provide a versatile toolkit for tackling different kinds of problems. Table 1.1 provides a summary of the core techniques.

Table 1.1. Comparison of Foundational Prompting Techniques

Technique	Description	Best Use Case
Zero-Shot	The model is given a task with no examples. It relies solely on its pre-trained knowledge.	General knowledge questions, simple summarization, creative text generation.
Few-Shot	The model is provided with 1-5 examples of the input/output pattern in the prompt.	Tasks requiring specific formatting, sentiment analysis, data extraction.
Chain-of-Thought (CoT)	The model is prompted to break down its reasoning into a series of steps before giving the final answer.	Complex logical reasoning, word problems, multi-step calculations.

Beyond these foundational methods, a wider array of techniques exists, as summarized in Figure ???. These advanced methods, such as role-playing and context-aware prompting, offer even finer control over the model's output.

1.2.6 Model Parameter Settings

Controlling the output of an LLM goes beyond the text of the prompt itself. The training covered the critical role of model parameters, which act as dials to tune the generative process:

- **Temperature:** This parameter adjusts the randomness of the output. A low temperature (e.g., 0.1) makes the model more deterministic and "safe," as it will always choose the most probable next word. This is ideal for factual tasks like data extraction. A high temperature (e.g., 0.9) increases randomness, allowing the model to be more creative and generate more diverse outputs, which is useful for brainstorming or creative writing.
- **Top-p (Nucleus Sampling):** This parameter provides another method for controlling randomness. Instead of considering all possible next words, Top-p considers only the smallest set of words whose cumulative probability exceeds the value of 'p'. This allows for a dy-

namic vocabulary size, leading to outputs that are often more natural than those produced by other methods.

- **Top-k:** This is a simpler sampling method where the model is restricted to choosing its next word from the 'k' most probable words. While effective, it can sometimes filter out surprisingly good but less probable word choices.

1.2.7 Advanced Prompting Architectures

The training also introduced system-level architectures that use prompting as a core component:

- **Retrieval-Augmented Generation (RAG):** RAG addresses a key limitation of LLMs: their knowledge is static and limited to their last training date. In a RAG system, a user's query is first used to search a private, up-to-date knowledge base (like a collection of company documents). The relevant information retrieved from this search is then injected into the prompt as context for the LLM. This enables the model to answer questions based on current, specific, and proprietary data, significantly improving factual accuracy.
- **AI Agents and Function Calling:** An AI Agent is an autonomous system that uses an LLM as its "brain" to reason, plan, and execute tasks. This is enabled by "function calling," a feature where the LLM can decide to invoke external tools (like a calculator, a search engine API, or a database query function). The LLM's role shifts from merely generating text to creating a plan of action and using available tools to accomplish a goal.

1.3 Software and Hardware Tools Learned

The training was software-centric, and no specialized hardware was necessary. The practical skills were developed using a suite of industry-standard tools:

- **Python:** As the lingua franca of AI development, Python was the primary language for the course. All API interactions, data processing, and application logic were written in Python, utilizing key libraries such as 'requests' for HTTP calls to model endpoints and 'json' for handling data.
- **Google AI Studio:** This web-based platform served as the primary environment for experimenting with Google's Gemini models. Its user-friendly interface allowed for rapid

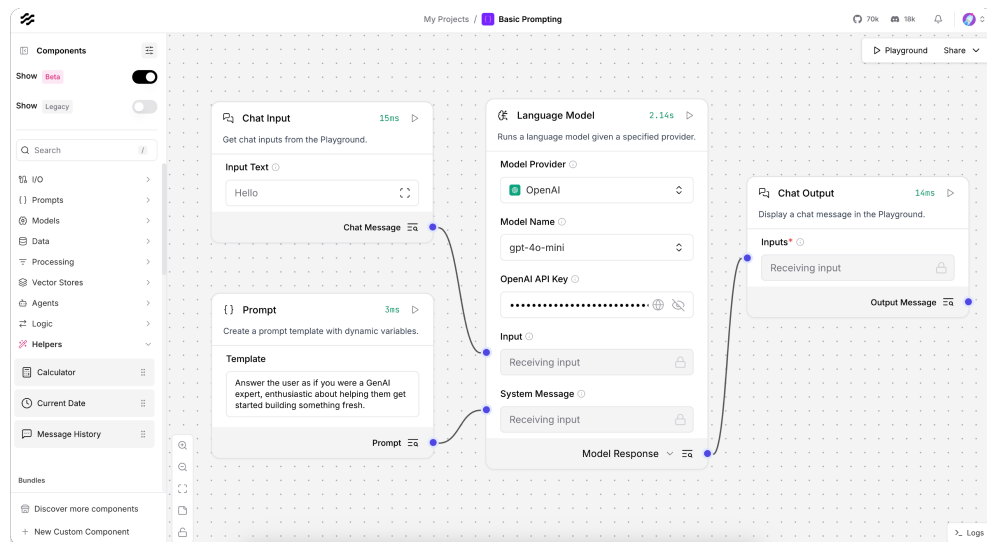


Figure 1.3. An example of the Langflow user interface, a visual tool used for building complex LLM applications by connecting different components.

testing of different prompt designs, adjustment of model parameters, and exploration of multimodal capabilities, including processing audio and speech inputs.

- **OpenAI Playground:** A similar interactive environment for OpenAI’s suite of models, including GPT-4. The Playground was used for comparative analysis and for tasks specific to the OpenAI ecosystem, such as preparing datasets and initiating fine-tuning jobs to create specialized models.
- **Langflow:** Langflow is a powerful, open-source graphical user interface for the LangChain framework. It was the central tool for the final week’s projects. By providing a visual, node-based editor, it allows developers to construct complex LLM applications by dragging, dropping, and connecting components like models, prompt templates, memory modules, and agents. This visual approach dramatically accelerates prototyping and debugging, making it an invaluable tool for applied AI development.

1.3.1 Key Applications

Prompt engineering is the enabling technology behind a vast array of NLP applications. The most critical applications where this skill is used include:

- **Language Translation:** By engineering prompts that specify the source and target languages, dialects, and stylistic nuances, LLMs can perform highly accurate and context-aware translations.

- **Question Answering Chatbots:** The effectiveness of a Q&A chatbot is almost entirely dependent on prompt engineering. By embedding user intent, context, and a relevant knowledge base into the prompt, chatbots can provide accurate and helpful answers, which is crucial for customer support and informational websites.
- **Text Generation:** For tasks like writing emails, reports, or creative content, prompts are used to define the purpose, tone, vocabulary, and structure of the generated text, ensuring it is fit for the intended audience and application.

CHAPTER 2

TRAINING WORK UNDERTAKEN

This chapter provides a sequential narrative of the practical work and learning milestones achieved during the four-week training program. The methodology was designed to be cumulative, with each week building directly upon the skills and concepts learned in the previous one.

2.1 Week 1: Foundational Skills and Concepts

The initial week was focused on establishing a robust theoretical and practical foundation. The objective was to move from a general understanding of AI to a specific, technical grasp of prompt engineering fundamentals, with an immediate emphasis on programmatic application using Python. The hands-on exercises were designed to demonstrate the power and flexibility of LLMs when controlled via code.

- **Code Summarization and Documentation:** The first practical exercise involved using Python to programmatically generate documentation for code snippets. The methodology involved writing a Python script that would take a function as a string input, embed it within a structured prompt, and then make an API call to an LLM. I experimented with different prompt templates to control the output's level of detail. A general prompt like, "Explain this code," was compared against more specific instructions such as, "Generate Python docstrings for this function, explaining its parameters, return value, and any exceptions it might raise." This exercise demonstrated the crucial concept of controlling the level of abstraction and showed that to effectively use LLMs for technical documentation, the prompt must precisely define the desired format and the intended audience.
- **Programmatic Image Generation:** The final exercise for the week focused on text-to-

image generation. I developed a Python script that accepted a descriptive text string as input (e.g., “a futuristic city at sunset”). The script then made an API call to an image generation model, passing this text as the prompt. The API response, which contained the image data, was then processed by the script and saved locally as a PNG file. This task provided hands-on experience with a different modality of generative AI. It taught me how to interact with APIs that return non-textual data and demonstrated how the principles of clear and descriptive prompting are just as crucial for generating high-quality images as they are for generating text.

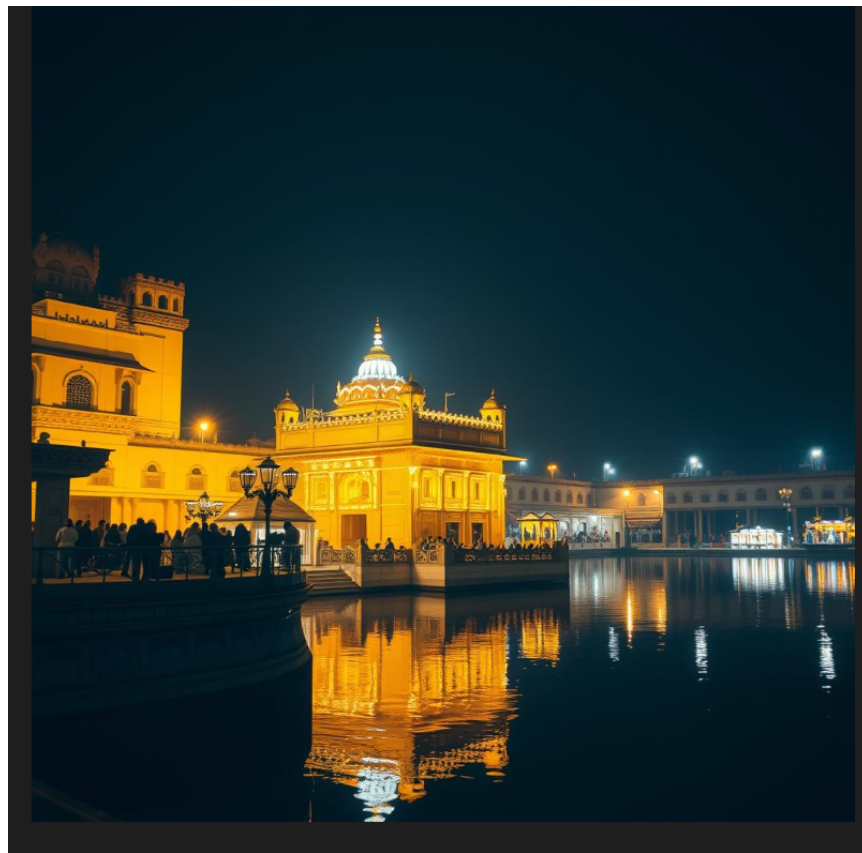


Figure 2.1. An image of the Golden Temple generated programmatically from a text prompt.

2.2 Project: Automated Web Content Summarizer

After covering the foundational exercises, the first significant project was to build a complete web summarizer application. The objective was to create a Python script that could take any URL as input and produce a concise summary of the webpage’s content. This project integrated several skills, including web scraping, text processing, and prompt engineering.

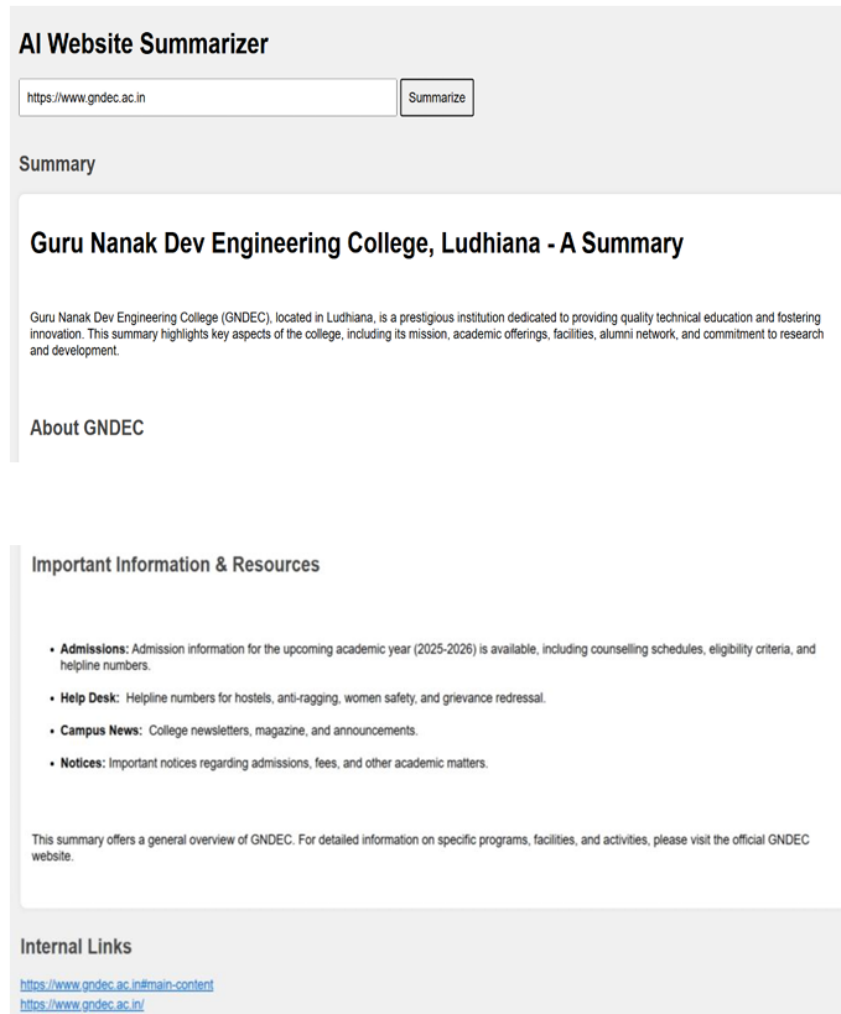


Figure 2.2. A screenshot showing the output of the Automated Web Summarizer, displaying the summary generated from the GNDEC college website.

2.2.1 Methodology and Learning

The process was broken down into a sequential pipeline:

1. **Fetching Web Content:** The first step was to retrieve the raw HTML content from the given URL. This was accomplished using the popular Python library 'requests'. This step taught me the basics of making HTTP GET requests and handling server responses.
2. **Parsing and Text Extraction:** Raw HTML is noisy and contains code, navigation links, and advertisements. To isolate the main content, I used the 'BeautifulSoup' library to parse the HTML document. I wrote functions to find and extract text from relevant tags like 'p' (paragraphs) and 'h1', 'h2' (headings), while discarding irrelevant content from scripts or navigation bars.
3. **Prompt Engineering for Summarization:** Once the clean text was extracted, it was

passed to an LLM. The prompt was carefully engineered for this task: “Please provide a concise, professional summary of the following article in three distinct bullet points. Focus on the main arguments, key findings, and overall conclusion.” This prompt design was chosen to ensure the output was structured, brief, and useful.

4. **API Integration and Output:** The final script assembled the prompt and the extracted text, sent it to an LLM API endpoint, and then printed the returned summary to the console.

This project was a critical learning experience. It was a practical demonstration of how LLMs can be connected to live, real-world data from the internet. The most important lesson was the principle of “Garbage In, Garbage Out”; the quality of the summary was directly dependent on how well the text was cleaned and prepared in the pre-processing steps.

2.3 Week 2: Prototyping with Google AI Studio and Multimodality

The second week transitioned from purely code-based interaction to using a high-level, web-based platform for rapid prototyping. The focus was on mastering Google AI Studio and exploring the advanced multimodal capabilities of the Gemini models.

2.3.1 Exploring the Google AI Studio Environment

The primary methodology for this week was exploratory and iterative. The initial task was to become proficient with the Google AI Studio interface, including creating new prompts, saving them, and understanding the model response panel. A key learning activity was the systematic testing of model parameters. I systematically ran the same prompt multiple times while adjusting the ‘Temperature’, ‘Top-p’, and ‘Top-k’ settings. This provided a practical intuition for how to tune a model for a specific task, as shown in Figure 2.3.

2.4 Week 2: Developing a Multimodal Voice Assistant

The second week transitioned from purely code-based interaction to the development of a more sophisticated, multimodal application. The focus was on integrating various AI capabilities to create a functional voice assistant that could handle Speech-to-Text, Text-to-Speech, and end-to-end Speech-to-Speech interactions, primarily using the Gemini models via API calls in a Python

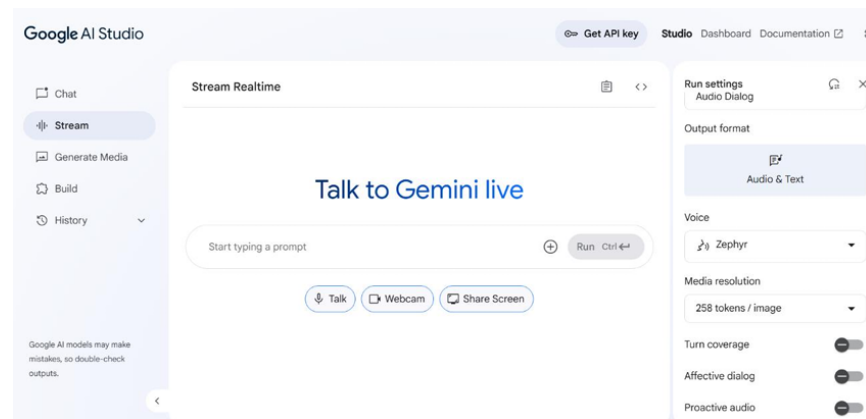


Figure 2.3. A screenshot of the Google AI Studio interface, showing the stream and chat functionalities for real-time interaction with Gemini models.

environment.

2.4.1 Project Architecture and Methodology

The main project for the week was the creation of a command-line voice assistant. The methodology was to build a modular system in Python where each core function (Text-to-Speech, Speech-to-Text) could be tested independently before being integrated into a final, interactive application. The assistant was designed with a simple menu-driven interface, as shown in Figure 2.4.

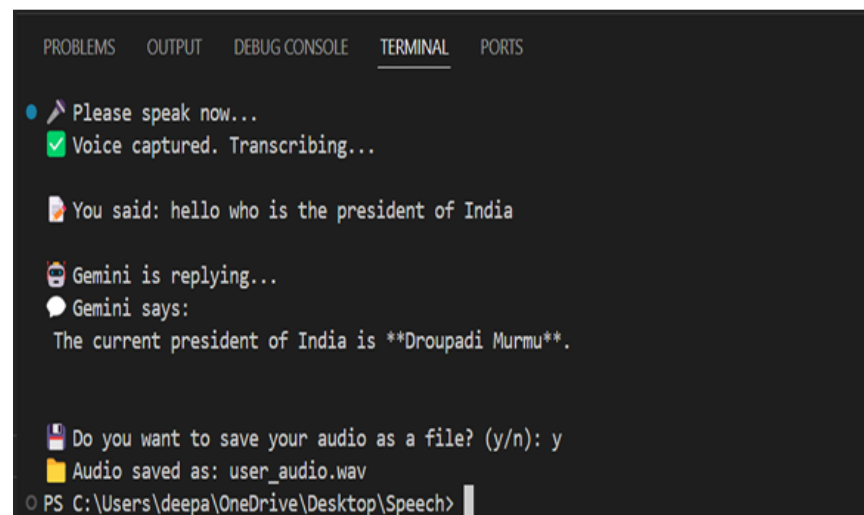


Figure 2.4. The menu-driven interface of the voice assistant, showing the options for Text-to-Speech, Speech-to-Text, and Speech-to-Speech.

2.4.2 Core Functionality Implementation

- **Text-to-Speech (TTS):** The first module implemented was TTS. A Python function was created that took a string of text as input, made an API call to a TTS model, and then

played the returned audio through the system's speakers. This was the foundational output mechanism for the assistant.

- **Speech-to-Text (STT):** The next module was STT. This involved writing a function that used a library like 'PyAudio' to capture microphone input for a set duration (e.g., 5 seconds). The captured audio data was then sent to a speech recognition API (like Gemini's), which returned the transcribed text. This served as the primary input mechanism.
- **Speech-to-Speech Integration:** The final and most complex task was to combine these modules into a seamless speech-to-speech loop. The methodology was as follows:
 1. The system prompts the user to speak.
 2. The STT module captures and transcribes the user's voice input (e.g., "who is the current president of India").
 3. The transcribed text is then used as a prompt for the Gemini language model.
 4. The LLM processes the prompt and generates a text-based answer (e.g., "The current president of India is Droupadi Murmu").
 5. This text answer is then fed into the TTS module, which speaks the response back to the user.

The final, integrated system provided a complete conversational turn, as demonstrated in Figure 2.5. This project was a significant learning experience in API integration and in orchestrating multiple AI services to create a single, cohesive application.

2.4.3 Key Learnings and Challenges

This project was a significant learning experience in system integration and in orchestrating multiple AI services. Several key engineering challenges were encountered and overcome:

- **API Latency:** The end-to-end response time was a noticeable factor, as the process involved three sequential API calls (STT, LLM, TTS). This highlighted the importance of efficient code and network speed for creating a responsive, real-time user experience.
- **Error Handling:** Robust error handling was implemented using 'try-except' blocks to manage potential issues such as API failures, network timeouts, or problems with microphone access, making the application more resilient.

```

Voice Assistant Menu
1. Text to Speech
2. Speech to Text
3. Speech to Speech
4. Exit
Enter your choice: 1
Enter text to speak: How are You?
Speaking: How are You?

Voice Assistant Menu
1. Text to Speech
2. Speech to Text
3. Speech to Speech
4. Exit
Enter your choice: 2
Recording for 5 seconds...
You said: who is the current president of India

Voice Assistant Menu
1. Text to Speech
2. Speech to Text
3. Speech to Speech
4. Exit
Enter your choice: 3
Recording for 5 seconds...
You said: who is the current president of India

```

Figure 2.5. The terminal output of the fully integrated Speech-to-Speech function, showing the transcribed user query and the LLM's generated response.

- **Environment Management:** The project required the careful management of several external Python libraries and API keys, providing practical experience in setting up and maintaining a development environment.

2.5 Week 3: Advanced Techniques with OpenAI and Agents

The third week marked a significant step up in complexity, focusing on customizing models for specific tasks and building more autonomous AI systems. The work transitioned to the OpenAI ecosystem to explore its unique features, such as model fine-tuning and advanced agentic behavior through function calling.

2.5.1 Project: Supervised Model Fine-Tuning

The first major project of the week was to create a supervised fine-tuned model. The objective was to specialize a base OpenAI model (such as GPT-3.5-Turbo) to become an expert at a specific task: converting natural language questions into SQL queries. This is a common and high-value use case for LLMs in enterprise environments.

Methodology

The process was a systematic, multi-step engineering workflow:

1. **Dataset Preparation:** The foundational step was the creation of a high-quality training dataset. I curated a dataset consisting of over 100 examples. Each example was a JSON ob-

ject containing a carefully crafted prompt (e.g., "Show me the total sales for the electronics department last month") and the ideal completion (the corresponding, syntactically correct SQL query). This process required careful consideration of various SQL commands, table structures, and query complexities.

2. **Data Formatting and Validation:** The dataset was formatted into the required JSON Lines (JSONL) format, where each line is a separate JSON object. A script was written to validate the dataset to ensure there were no formatting errors that would cause the fine-tuning job to fail.
3. **Initiating the Fine-Tuning Job:** Using the OpenAI Python library, a script was developed to upload the validated dataset and initiate the fine-tuning job. This involved selecting the base model and specifying training parameters.
4. **Evaluation:** After the job was completed, the performance of the new, fine-tuned model was rigorously evaluated. It was tested against a held-out set of 20 natural language questions. Its performance was compared directly against the base model. The fine-tuned model demonstrated a significantly higher accuracy in generating correct SQL queries, especially for questions involving complex joins and aggregations.

2.5.2 Developing AI Agents with LangChain and Function Calling

The second focus of the week was on building AI Agents. An agent is a system that uses an LLM as a reasoning engine to decide which actions to take and in what order. This was achieved by integrating LLMs with external tools through a framework like LangChain and leveraging the underlying model's function calling capability.

Methodology

The project was to build a simple agent that could answer questions about current events by using a real-time news search tool. The methodology was as follows:

1. **Tool Definition:** A simple Python function was defined to act as the "tool." This function would take a search query string as input and use an external News API to find and return relevant article headlines.

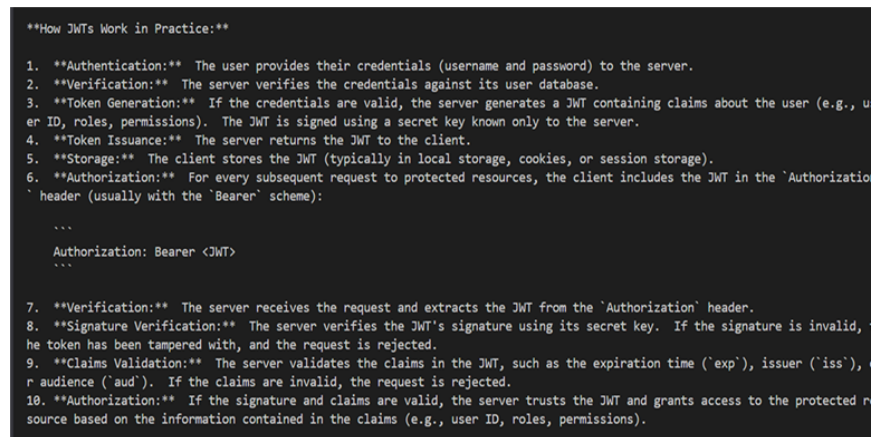


Figure 2.6. A diagram illustrating the workflow of an AI Agent using function calling to interact with external tools.

2. **Agent Initialization:** Using the LangChain library, an agent was initialized. This involved providing the agent with two key components: the LLM to be used as its reasoning "brain," and the set of tools it was allowed to use (in this case, just the news search tool).
3. **Agentic Loop:** When the user asked a question (e.g., "What are the latest developments in AI regulation?"), the agent's LLM would recognize that it could not answer from its internal knowledge. It would then decide to use the news search tool. The agent would determine the correct input for the tool (e.g., the query "AI regulation developments") and execute it.
4. **Execution and Response:** The LangChain framework handles the execution of the tool and captures its output (the news headlines). This output is then fed back to the LLM as new context. With this up-to-date information, the LLM generates a comprehensive and accurate final answer for the user.[2]

This project was a powerful demonstration of how to extend the capabilities of an LLM beyond its pre-trained knowledge, allowing it to interact with the live, external world through a structured agentic framework.

2.6 Week 4: Capstone Project - The AI Research Assistant

The final week was the culmination of the training, synthesizing all learned concepts into a single, full-stack web application: an AI Research Assistant. This project moved beyond simple scripts to a complete, user-facing product with a database, user authentication, and multiple, integrated AI functionalities. The application was built using the Flask web framework for the

backend, SQLite for the database, and the Gemini API via the LangChain library for the core AI capabilities.[3]

2.6.1 System Architecture and Features

The application was designed with a robust architecture to support multiple features for authenticated users, as shown in the screenshots below.

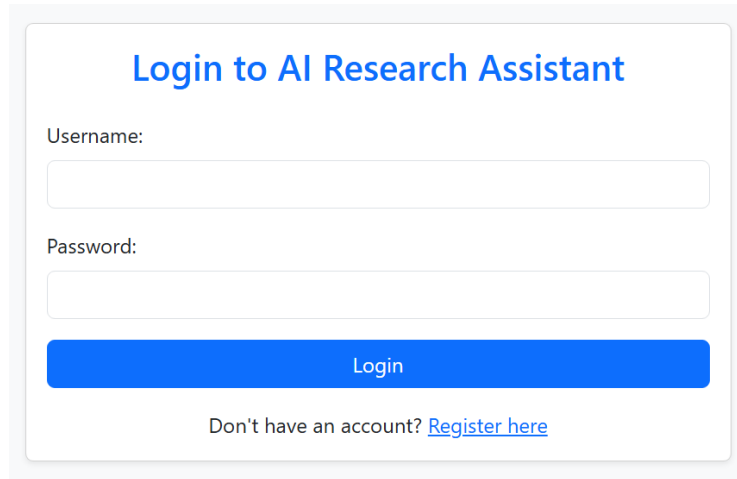


Figure 2.7. The secure login page for the AI Research Assistant, built using Flask.

Key components of the system included:

- **User Authentication:** A complete user registration and login system was implemented using a SQLite database and Flask-JWT-Extended to manage secure, cookie-based user sessions.
- **Multi-Source Summarizer:** A versatile tool that allowed users to generate detailed summaries from a simple topic, a URL, or an uploaded PDF document.
- **Interactive PDF Chat (RAG System):** The core feature was a powerful Retrieval-Augmented Generation (RAG) system that allowed users to have an in-depth conversation about the contents of an uploaded PDF document.[4]
- **User History:** All generated summaries were saved to the database, linked to the user's account, allowing them to view their past research, as seen in Figure 2.9.

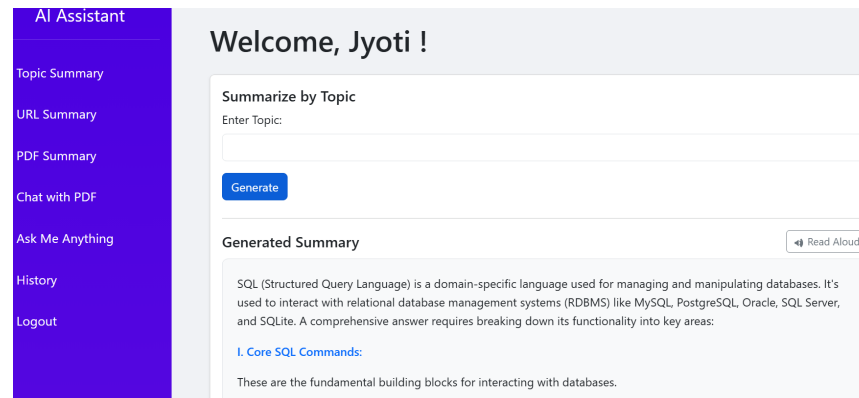


Figure 2.8. The main dashboard of the AI Assistant after a successful login, showing the various summarization and chat features available to the user.

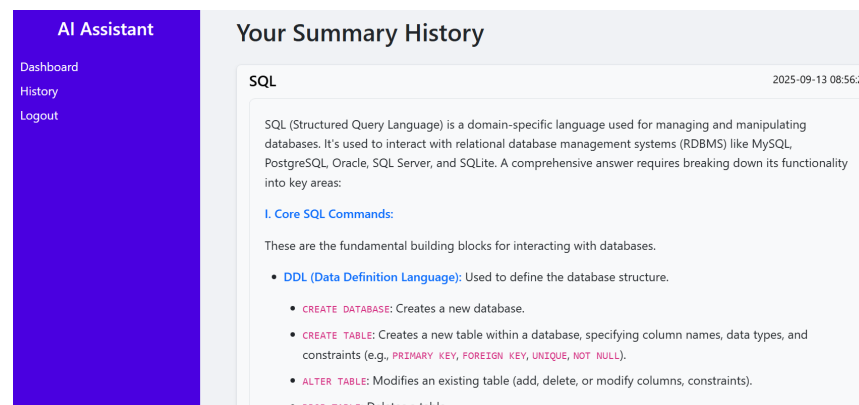


Figure 2.9. The user's summary history page, demonstrating the application's ability to save and retrieve past results from the SQLite database.

2.6.2 Methodology: The RAG Pipeline for PDF Chat

The implementation of the PDF Chat feature was the most technically complex part of the project. The workflow was divided into two main stages: document ingestion and question answering. This was achieved using key LangChain components such as 'PyPDFLoader' for text extraction, 'CharacterTextSplitter' for chunking, 'GoogleGenerativeAIEmbeddings' for vectorization, and 'FAISS' for creating a searchable vector store. This powerful pipeline allowed the application to provide accurate, context-aware answers based solely on the content of the uploaded document.

[5]

CHAPTER 3

RESULTS AND DISCUSSIONS

This chapter presents a detailed analysis of the results obtained from the capstone project, the AI Research Assistant. The discussion focuses on the qualitative performance of the application's key features, the effectiveness of the chosen methodologies, and the engineering challenges and learnings that emerged during its development.

3.1 Performance of the Multi-Source Summarizer

The summarization tool was a core feature of the application, designed to handle three distinct types of input. Its performance was evaluated qualitatively for each type.

3.1.1 Results

- **Topic Summary:** When provided with a general topic, such as "SQL," the system leveraged the Gemini model's search-enabled generation capabilities. As shown in the dashboard screenshot (Figure 2.8), the resulting summaries were comprehensive, well-structured, and factually accurate, providing a detailed overview of the topic.
- **URL Summary:** The performance of the URL summarizer was largely dependent on the structure of the target website. For well-structured articles and blog posts, the 'BeautifulSoup' text extraction was effective, leading to high-quality summaries. However, for highly dynamic, JavaScript-heavy websites, the text extraction was sometimes incomplete, which in turn affected the quality of the summary.
- **PDF Summary:** The PDF summarizer, which utilized a detailed, hierarchical prompt template, produced excellent results. It successfully generated in-depth, well-organized

summaries of complex documents, capturing key sections, definitions, and conclusions as intended. The instruction to ignore preliminary sections like the table of contents was followed correctly by the model.

3.2 Performance of the RAG System (PDF Chat)

The interactive PDF chat was the most advanced feature of the AI Research Assistant. Its performance is a direct measure of the effectiveness of the implemented Retrieval-Augmented Generation (RAG) pipeline.

3.2.1 Results

The system's ability to answer questions based on a private knowledge base was highly successful.

- **Accuracy and Factuality:** The RAG system consistently provided accurate answers that were grounded in the text of the uploaded document. The prompt's strict instruction to answer *only* from the provided context was crucial in preventing the model from hallucinating or introducing external information. For example, when asked a question whose answer was not in the document, the model would correctly state that the information was not available in the provided text.
- **Contextual Understanding:** The system demonstrated a strong ability to find and synthesize information from different parts of the document to answer a single query. For instance, when a research paper was uploaded, the assistant could answer specific, detailed questions such as, "What were the limitations of the study mentioned by the authors in the conclusion?" or "What was the specific accuracy reported in Table 3 of the results section?" This indicated that the similarity search within the FAISS vector store was effectively retrieving the most relevant context for the LLM to use.

3.3 Discussion of Challenges and Key Learnings

The development of the AI Research Assistant provided several important engineering insights and challenges.

3.3.1 Challenges

- **Data Pre-processing (Chunking):** A key challenge in the RAG pipeline was determining the optimal strategy for splitting the document into chunks. The ‘CharacterTextSplitter’ was effective, but finding the ideal ‘chunk_size’ and ‘chunk_overlap’ was an iterative process. If chunks were too large, the context provided to the model could be noisy and contain irrelevant information. If they were too small, the context could be too fragmented, and the model might miss the broader picture. A ‘chunk_size’ of 1000 characters with an overlap of 200 was found to be a good balance for the documents tested.
- **System Latency:** The document ingestion process, which involves loading, chunking, and creating embeddings for an entire PDF, was computationally intensive and introduced noticeable latency for larger files. This is a common challenge in RAG systems.
- **Prompt Engineering for RAG:** Crafting the final prompt for the QA chain was critical. Early, simpler prompts sometimes resulted in the model reverting to its general knowledge. Adding a very strict instruction, such as ”You are a helpful assistant. Answer the question using ONLY the context below. If the answer is not in the context, say: ’The answer is not available in the document.’”, was essential for ensuring the reliability and factuality of the system.

3.3.2 Key Learnings

- **The Power of LangChain:** The LangChain library proved to be an invaluable tool for orchestrating complex AI pipelines. It provided robust, pre-built components for every step of the RAG process, from document loading to question answering, which significantly accelerated the development process.
- **The Importance of Grounding:** This project was a powerful demonstration of the importance of ”grounding” LLMs with external data. The RAG architecture is a highly effective pattern for building AI applications that are not only intelligent but also trustworthy and factual.
- **Full-Stack Integration:** Integrating the AI backend with a Flask web server, a database, and user authentication provided a comprehensive, end-to-end software engineering experience.

rience. It highlighted that building a real-world AI application involves much more than just writing prompts; it requires a solid understanding of web development, database management, and system architecture.

CHAPTER 4

CONCLUSION AND FUTURE SCOPE

This chapter consolidates the knowledge and experience gained throughout the industrial training, summarizing the key conclusions and outlining potential directions for future learning and development in the rapidly advancing field of Prompt Engineering.

4.1 Conclusion

This one-month industrial training provided a deep and comprehensive immersion into the theory and practice of Prompt Engineering. The program successfully bridged the gap between a high-level understanding of Artificial Intelligence and the specific, technical skills required to build valuable applications using Large Language Models. The curriculum effectively demonstrated that interacting with modern AI is a sophisticated discipline that extends far beyond casual conversation, requiring a systematic approach to instruction, design, and evaluation.

The key conclusion drawn from this experience is that the prompt is the primary user interface for the next generation of software. The ability to craft clear, contextual, and unambiguous prompts is the most critical skill for harnessing the power of LLMs. The training culminated in the development of a full-stack AI Research Assistant, which served as a practical demonstration of these principles. Building this application, from the user authentication layer to the complex RAG pipeline for PDF interaction, proved that with carefully engineered prompts and a well-designed system architecture, it is possible to create powerful, automated solutions to real-world problems. This training has provided a robust and immediately applicable skill set, solidifying my understanding of applied AI and providing a strong foundation upon which to build a career in this exciting field.

4.2 Future Scope

The field of generative AI is characterized by its rapid pace of innovation. While this training provided a comprehensive snapshot of the current state-of-the-art, continuous learning is essential. Based on the insights gained from the projects, several areas present compelling opportunities for future work and deeper exploration.

- **Advanced RAG Techniques:** The RAG system built for this project used a basic "stuff" chain. Future work could involve exploring more sophisticated techniques like 'Map-Reduce' or 'Refine' chains, which are better suited for processing and querying extremely large documents that do not fit into a single context window.
- **Development of Autonomous AI Agents:** The training provided a glimpse into AI agents. The future scope involves building more sophisticated and autonomous agents that can perform complex, multi-step tasks. This includes exploring advanced frameworks like ReAct (Reason + Act), which enable agents to create and execute complex plans, interact with multiple APIs simultaneously, and learn from their actions.
- **Multimodal Model Applications:** While the training touched upon multimodal inputs, the field is rapidly expanding to include a deeper fusion of text, images, and video. Future projects could focus on building applications that can analyze and generate content across these modalities, such as creating systems that can answer questions about images or generate video storyboards from a text script.
- **AI Safety, Ethics, and Alignment:** As AI systems become more powerful and autonomous, ensuring they are safe, ethical, and aligned with human values is paramount. A critical area for future study is the technical side of AI safety, including topics like prompt injection, jailbreaking, and methods for detecting and mitigating model bias. Developing skills in this area is not just a technical pursuit but a professional and societal responsibility.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems* 30, Long Beach, USA, 2017, pp. 5998–6008.
- [2] A. Chase. (2024). *LangChain Documentation* [Online]. Available: <https://python.langchain.com/>
- [3] A. Ronacher. (2024). *Flask Documentation (Version 3.0.x)* [Online]. Available: <https://flask.palletsprojects.com/>
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems* 33, Vancouver, Canada, 2020, pp. 9459–9474.
- [5] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Information Theory*, vol. 67, no. 8, pp. 5472–5489, Aug., 2021.